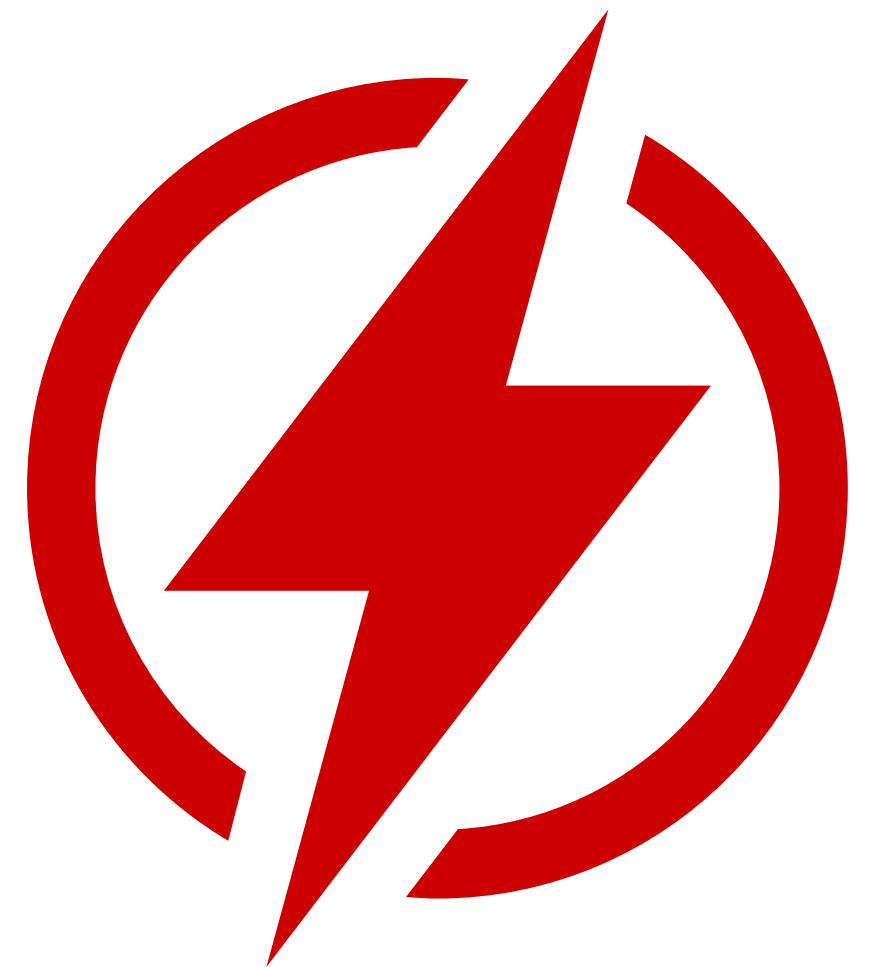


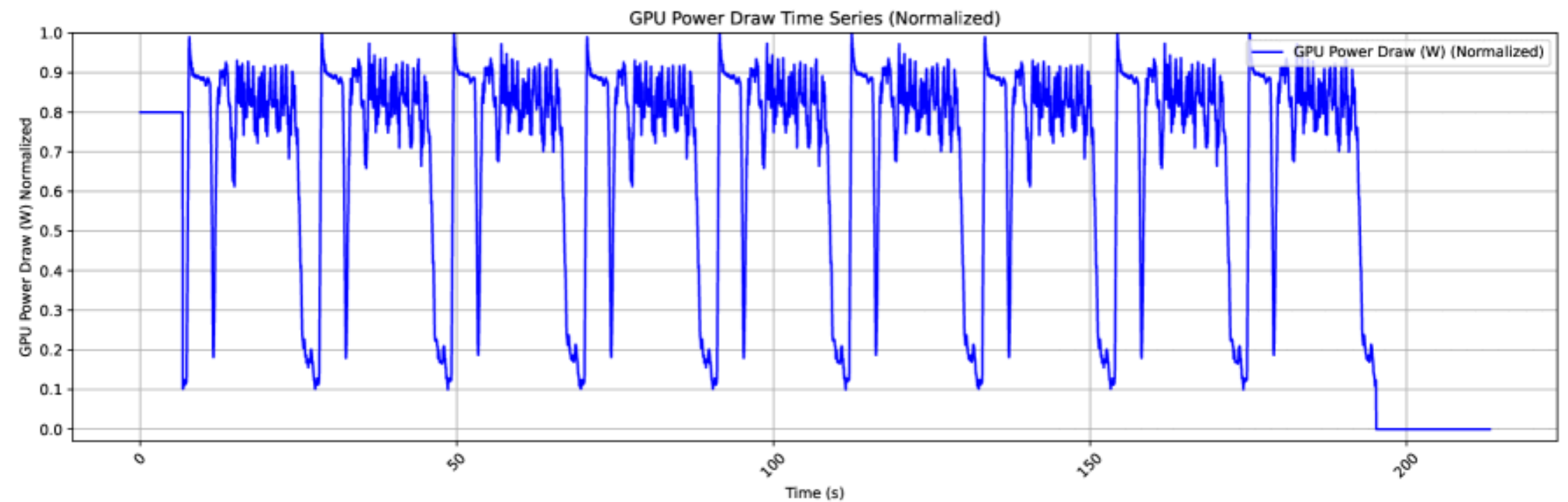
COLLECT & ADJUST SERVER LEVEL POWER CONSUMPTION

Emily, Alex, Daniel

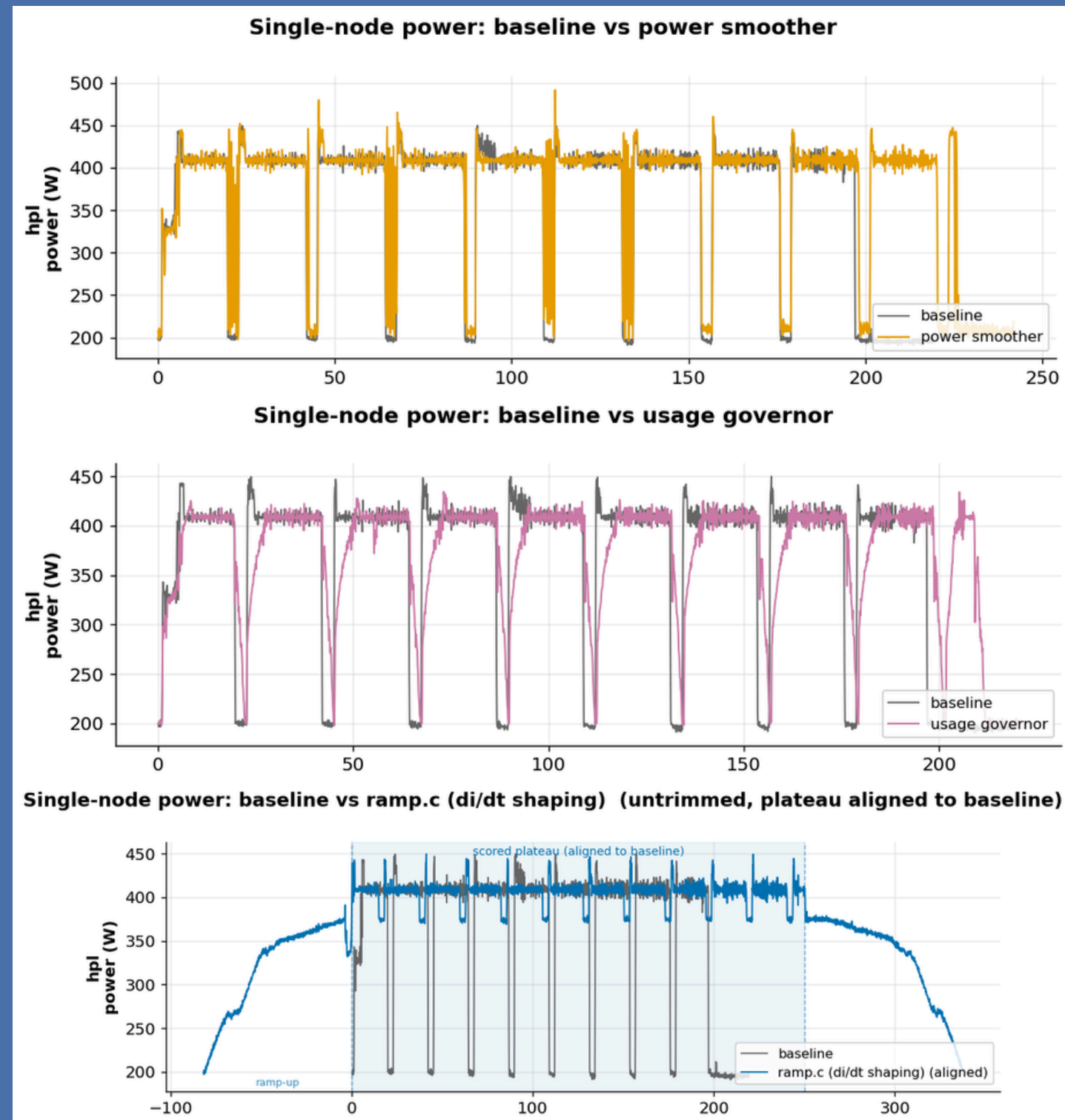


THE PROBLEM

- HPC jobs → Power fluctuations
 - Synchronous (AI training)
- Grid impacts
 - damage to hardware, etc



OUR SOLUTIONS



- 3 distinct mechanisms
 - “smoothers”
- **Goal:** Manage abrupt power swings
 - Rise → “spikes”
 - Limit increase
 - Ramp up
 - Fall → “drops”
 - Burn power
 - Ramp down

RAPL & REDFISH

- For server power:
 - BMC Redfish
 - RAPL
 - High freq polling
- Power capping

Table 2. Telemetry Comparison

	RAPL	Redfish
Scope	CPU package, cores, DRAM	Whole chassis
Granularity	Per-socket, per-domain	Chassis total
Useful rate	~10 Hz	~1 Hz
Path	In-band, host CPU	Out-of-band, BMC
Host overhead	<3% of one core	None
Requires	Root on host	BMC credentials + network
Survives host hang	No	Yes
Capping	Yes — millisecond-window PL1/PL2	Yes — coarse, slow (DCMI)
Portability	x86, vendor-specific	Vendor-neutral standard

MONITORING POWER & HARDWARE HEALTH

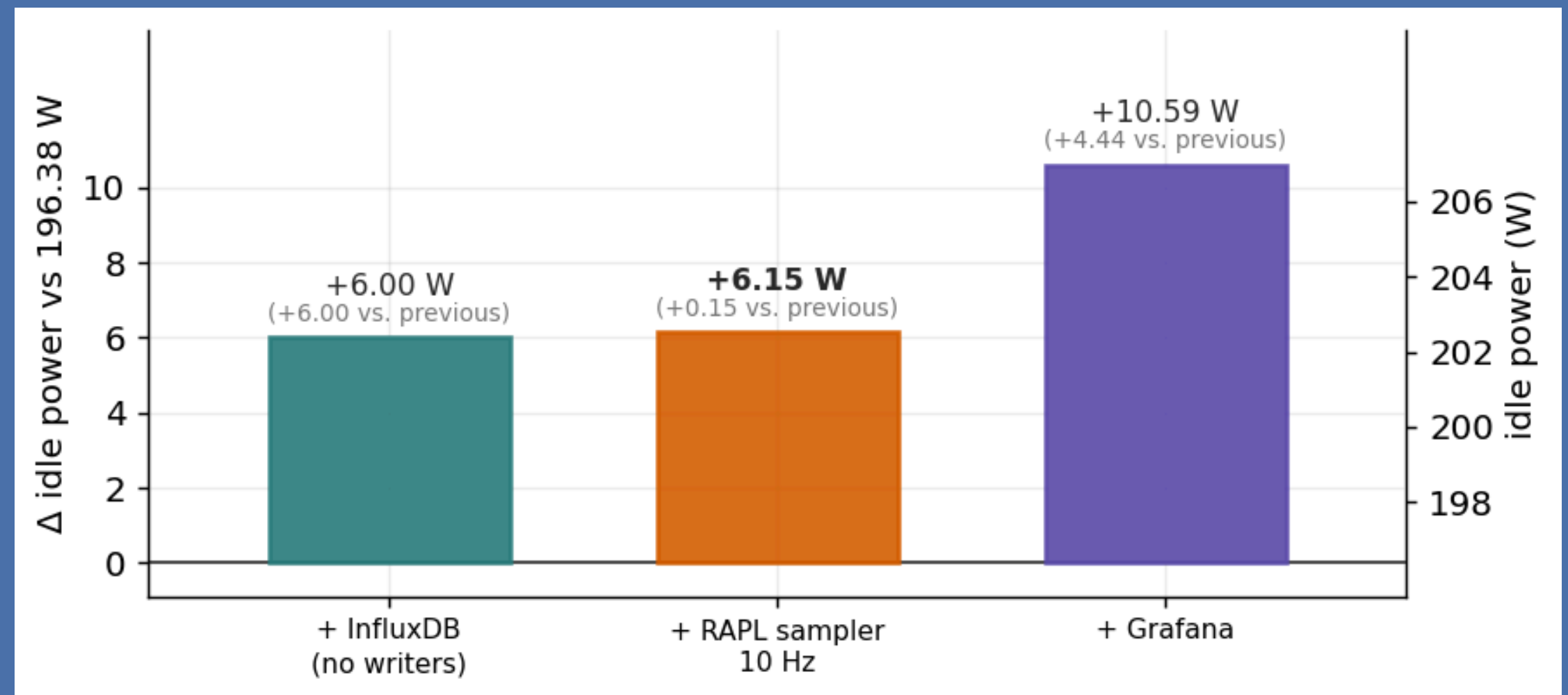
- Collect data from
 - Linux system telemetry
 - e.g. /proc/stat
 - PDU
 - Redfish
- Daemons

LIVE DASHBOARD



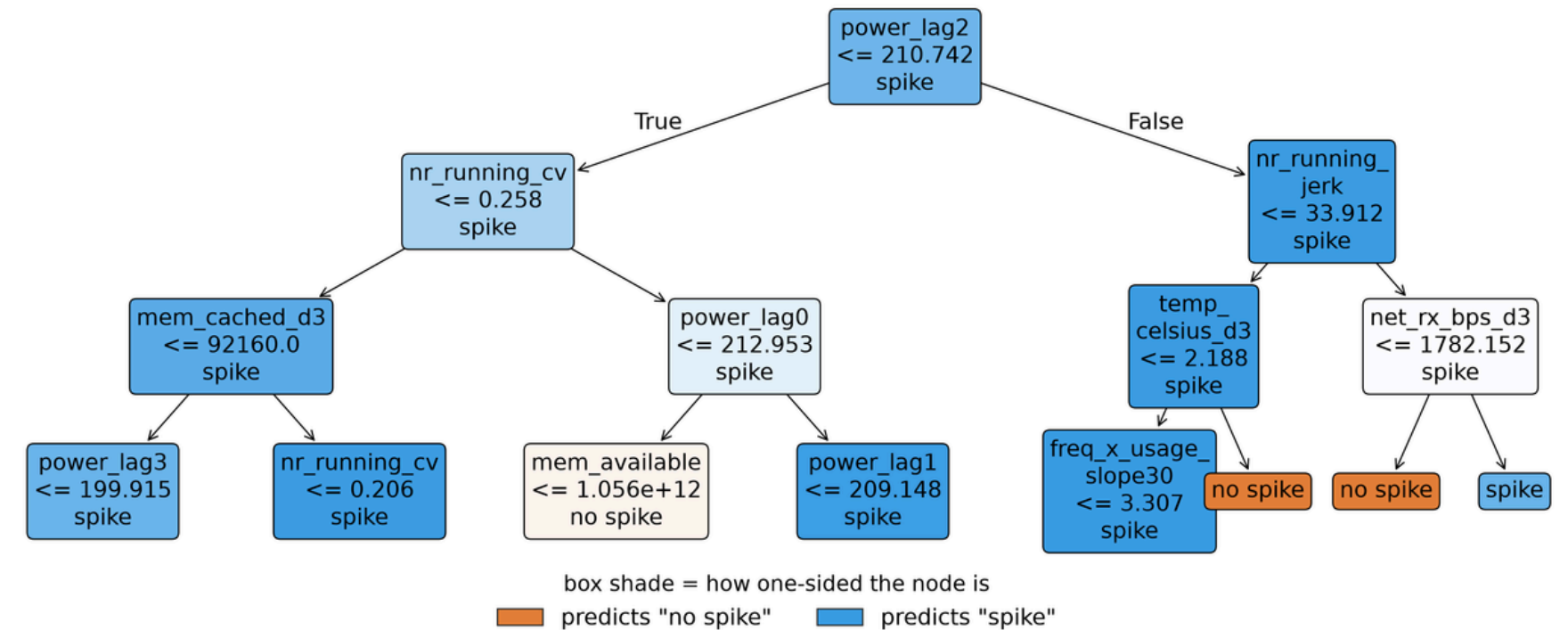
THE STACK

- Live graphs
- Data → InfluxDB → Grafana
- Background daemons
 - RAPL
 - BMC Redfish
 - PDU
 - Linux system telemetry



MACHINE LEARNING APPROACHES

- Predict swings → smoothing
- Random Forest + KF
 - RF → unbiased
 - calculate risk, threshold
 - KF cleans signal



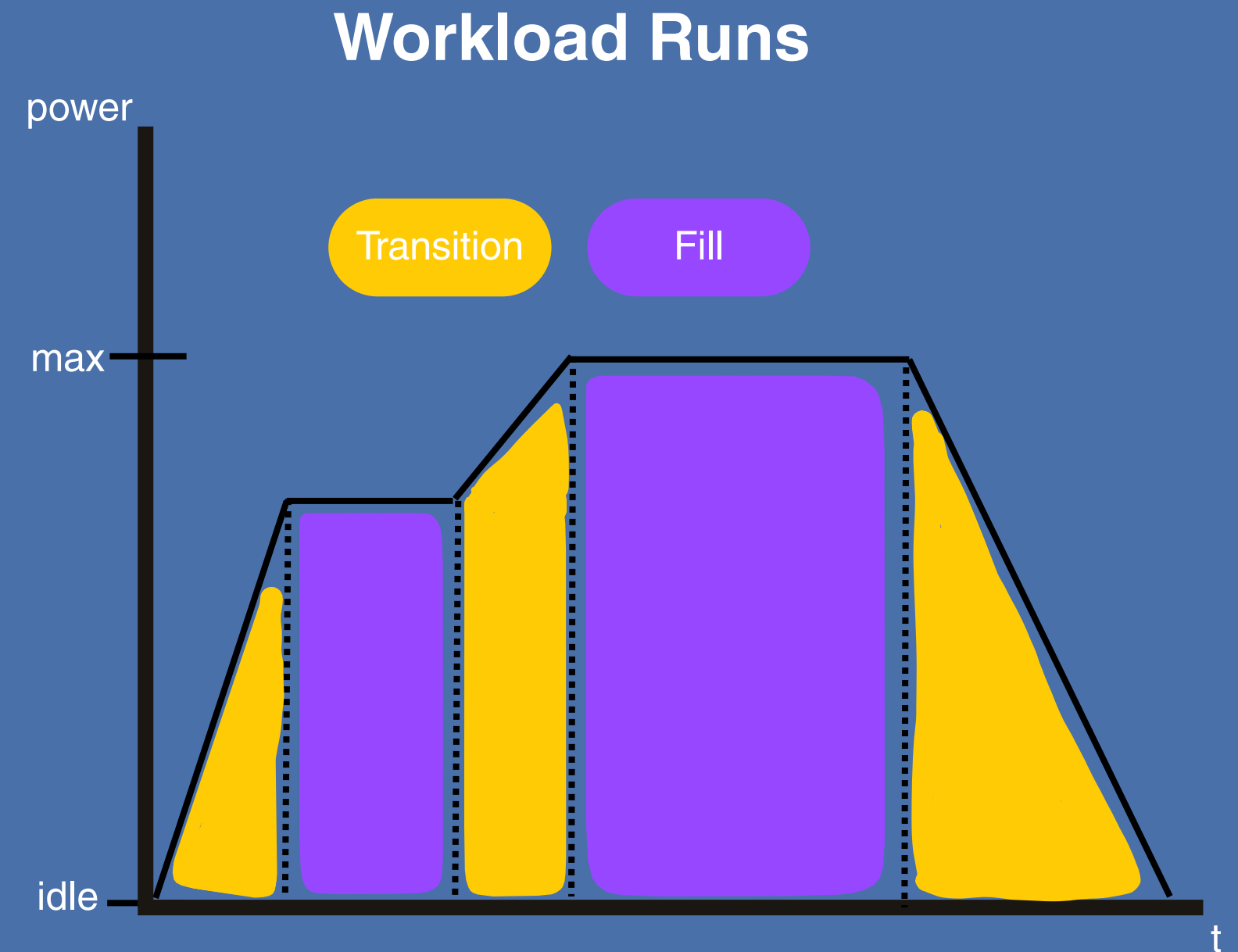
- Hardware limit for latency
 - Prediction → Detection
- Continual training, upstream data, quant finance methods, etc...
- Usage-based detector (usage_edge) outperformed every ML model

THE SMOOTHERS

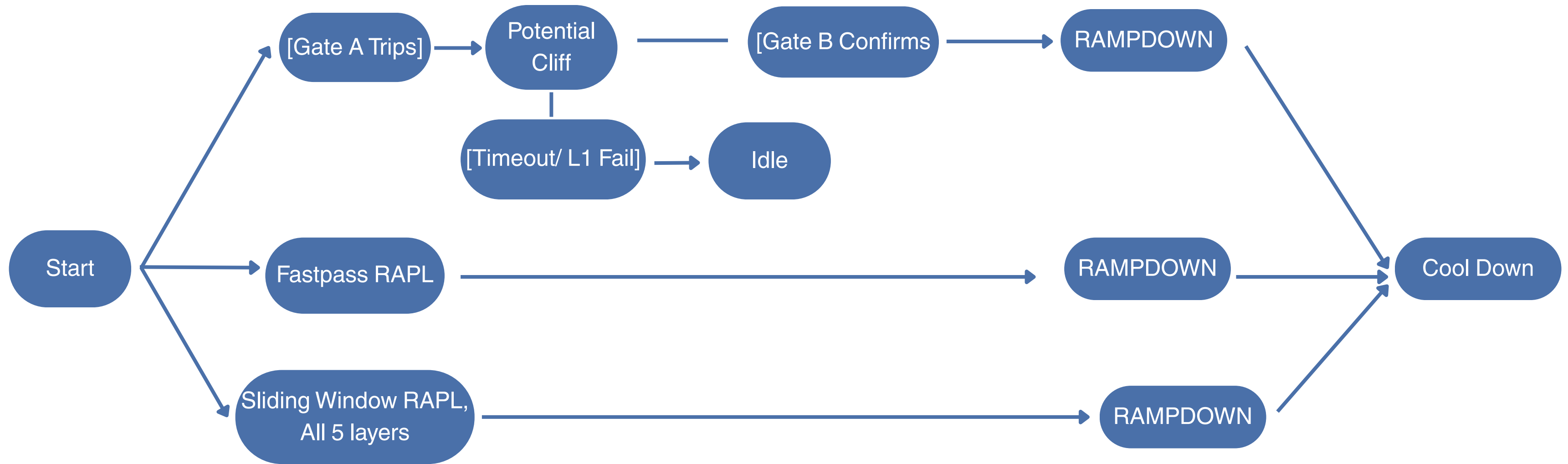
- Ramp.c
 - Trapezoid
- Ramp Down
- Governor
 - Detect swings
 - Ramp up & down

RAMP.C

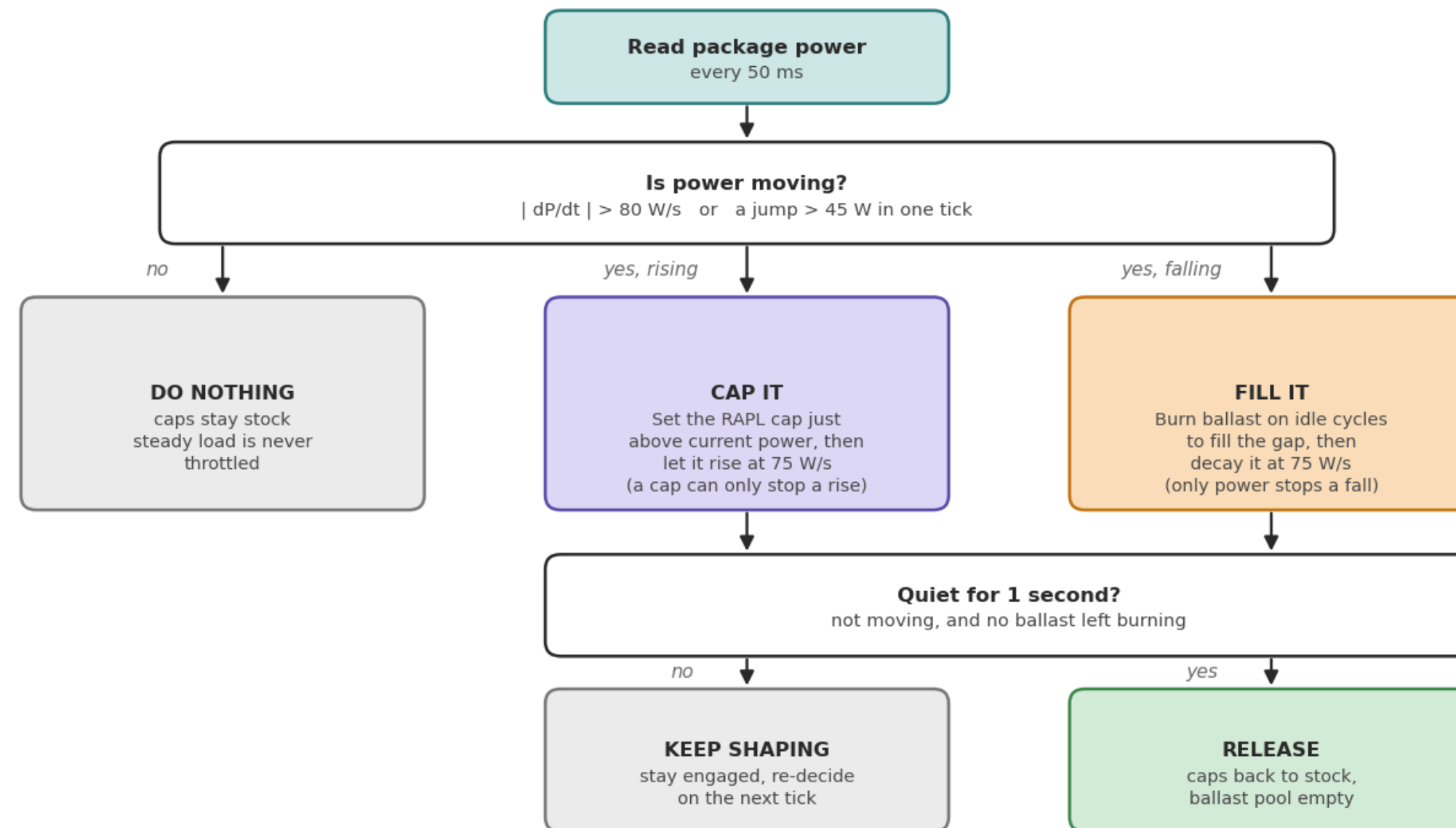
- Process:
 - Ramp power max
 - Run workload
 - Ramp down
- Manager processes
 - Controls workers
- Transitioning
 - Between primary tasks
- Brute force approach (eats cost!)



RAMP DOWN PROCESS



GOVERNOR



One tick. It repeats 20 times a second.

- usage_edge to detect swings
- Spike → RAPL powercap
- Drop → ballast
 - scheduler assigns duty-cycled jobs from a pool
- Strict swing definition
 - Prone to Type II errors

DATA COLLECTION

Run matrix

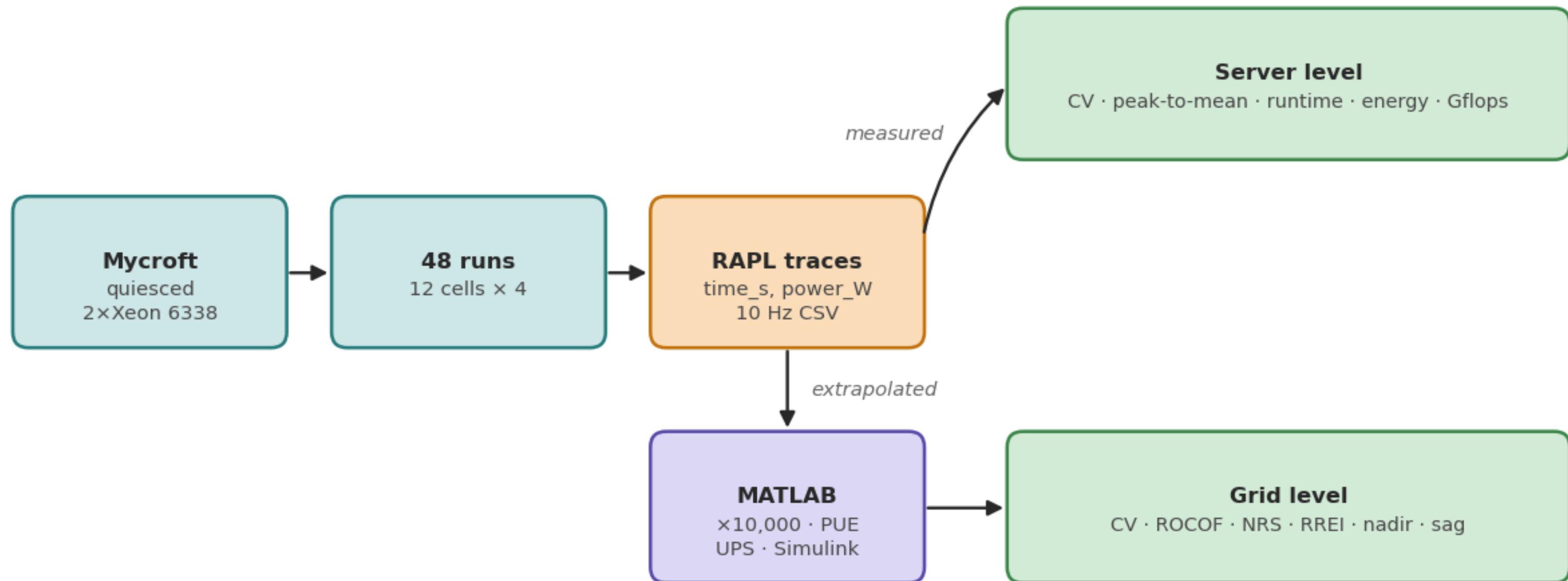
	baseline	power smoother	ramp.c	usagegov
HPL (ai_load.sh)	×4	×4	×4	×4
AI Sim 2	×4	×4	×4	×4
Step	×4	×4	×4	×4

3 workloads × 4 conditions × 4 repeats = 48 traces

Each cell

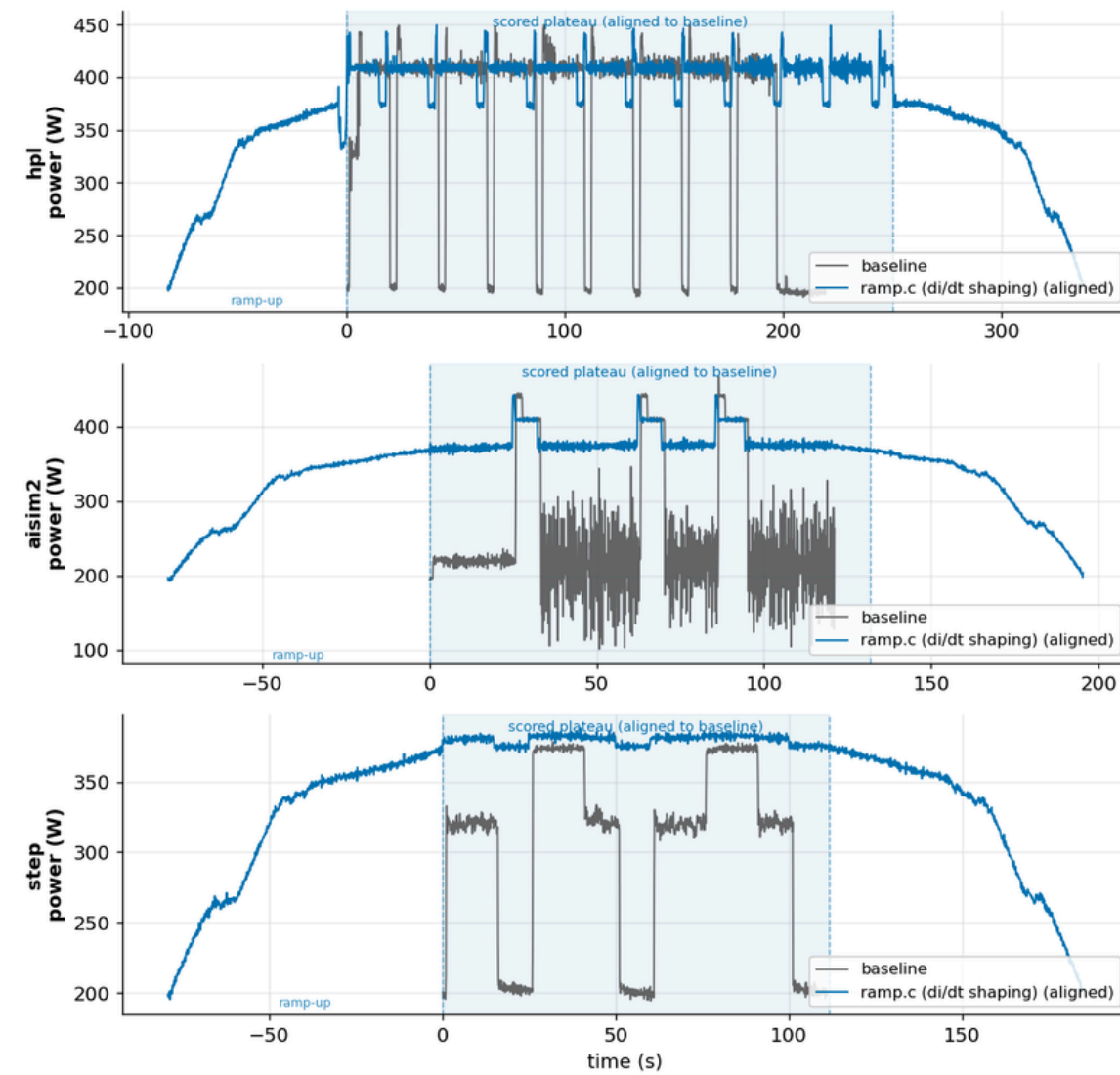


ANALYSIS PIPELINE

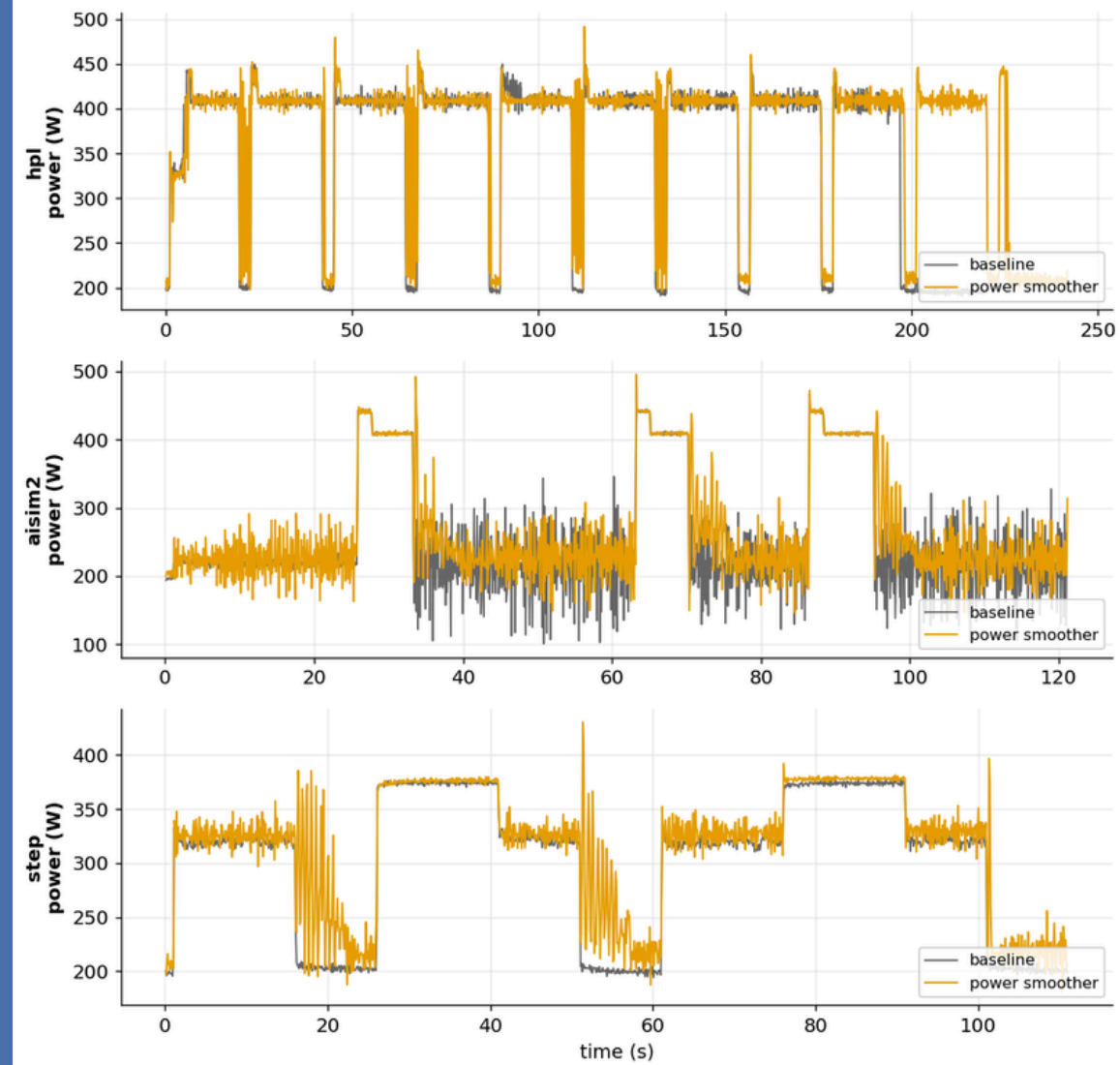


RESULTS

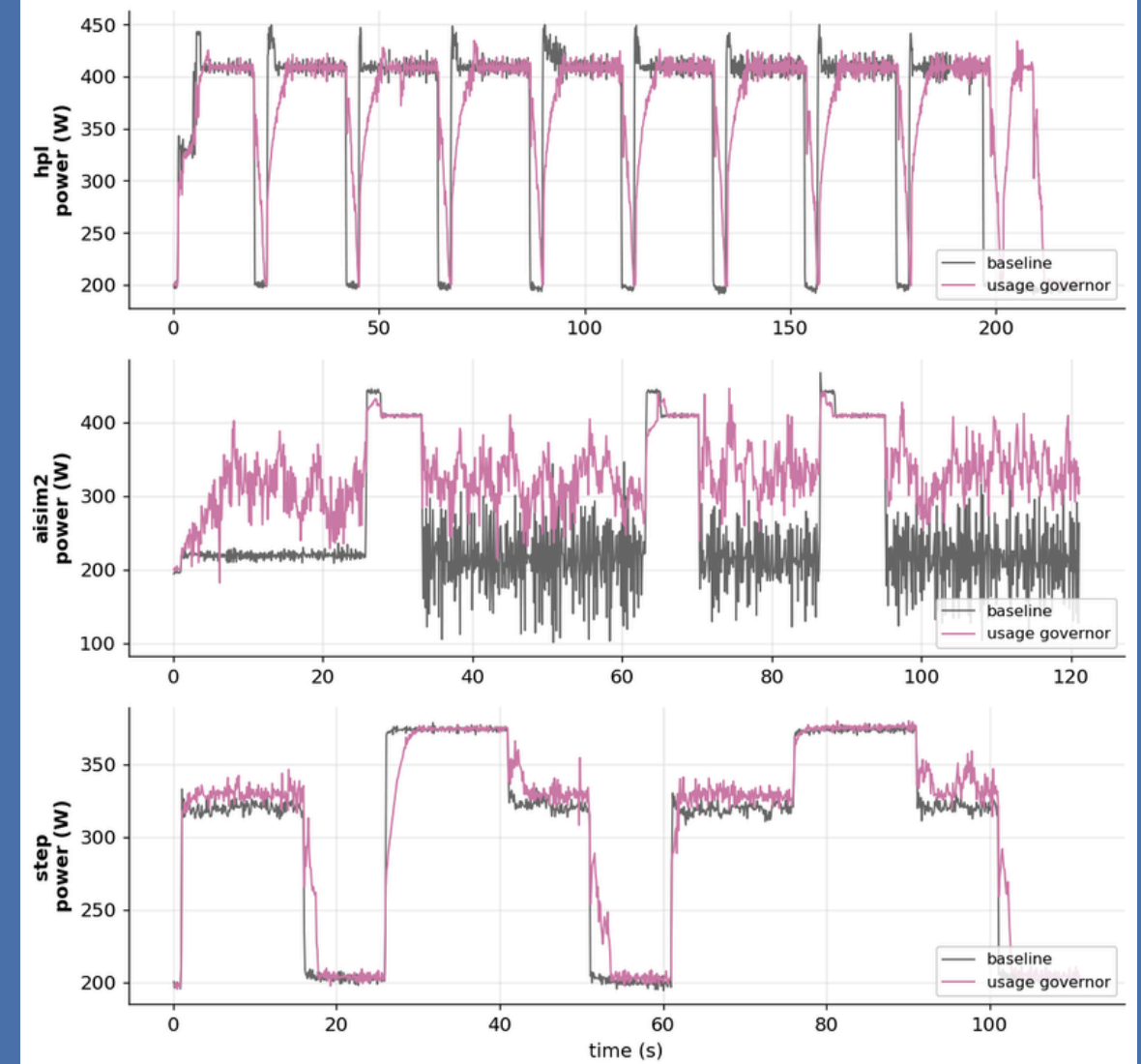
Single-node power: baseline vs ramp.c (di/dt shaping) (untrimmed, plateau aligned to baseline)



Single-node power: baseline vs power smoother



Single-node power: baseline vs usage governor



CONCLUSION

- Inconclusive results
 - Limited by time, technical skills
- All 3 take promising approaches
 - Future research → consolidate logic

